

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Cloud Computing (CS2032)

Proyecto: CloudCommerce - Plataforma de comercio electrónico basada en microservicios y arquitectura cloud

Integrantes Grupo:

Daniel Guillermo Sandoval Toro (202310533)

Jose Ernesto Guerrero Cueva (202210566)

Ian Jesus Rodríguez Santayana (202310184)

Vannia Fatima Curay Mauricio (201910133)

Mia Wood De la fuente chavez (202410085)

Profesor:

Mejia Fernandez, Oscar Rodolfo

ÍNDICE

1. INTRODUCCIÓN	3
1.1. Objetivos.....	3
1.2 Correspondencia con la rúbrica.....	3
2. ARQUITECTURA DE SOLUCIÓN.....	4
2.1. Componentes y servicios AWS.....	4
2.2. Recorrido de una petición.....	5
2.3. Decisiones de diseño y seguridad.....	5
3. BACKEND: MICROSERVICIOS.....	6
3.1. Resumen.....	7
3.2. Microservicio de productos.....	7
3.3. Microservicio de usuarios.....	8
3.4. Microservicio de pedidos.....	8
3.5. Microservicio de checkout.....	8
3.6. Microservicio de analítica.....	9
3.7. Modelo de datos.....	10
3.8. Pruebas y documentación de las API.....	11
4. FRONTEND.....	12
5. DATA SCIENCE.....	16
5.1. Ingesta hacia S3.....	16
5.2. Catálogo de datos.....	17
5.3. Consultas y vistas en Athena 17.....	18
6. DESPLIEGUE EN AWS 21.....	22
7. LIMITACIONES Y TRABAJO FUTURO 23.....	24
8. CONCLUSIONES.....	24

1. INTRODUCCIÓN

CloudCommerce es un e-commerce simple construido como proyecto parcial del curso. Su propósito es demostrar el diseño, despliegue y análisis de una solución en la nube: un backend en 5 microservicios con tecnologías y bases de datos distintas, un frontend web que los consume, y un pipeline de Data Science que lleva los datos transaccionales a un data lake en S3 para consultarlos con Athena.

1.1 Objetivos

- Implementar microservicios REST independientes (Python, Java y Node.js) con bases de datos propias: MySQL, PostgreSQL y MongoDB.
- Desplegar el backend en una arquitectura de producción con balanceador de carga interno, API Gateway y bases de datos en una máquina virtual privada.
- Publicar una aplicación web en AWS Amplify que consuma los cinco microservicios.
- Ingerir el 100 % de los datos hacia S3, catalogarlos con AWS Glue y analizarlos con consultas y vistas en Athena.

1.2 Correspondencia con la rúbrica

Ítem	Pts.	Dónde se documenta
Backend: microservicios	7	Sección 3 (más sección 6, despliegue)
Frontend: web	3	Sección 4
Data Science	5	Sección 5
Diagrama de Arquitectura de Solución	1	Sección 2, Figura 1
Exposición presencial	1	Presentación (archivo aparte)
Exposición virtual con ACL	3	Hito 1 (fuera de este informe)

"El código completo y las guías de despliegue están en el repositorio público de GitHub: <https://github.com/DaniSandt1/CloudCommerce> (README.md, DEPLOYMENT.md y el README de cada carpeta)."

2. ARQUITECTURA DE SOLUCIÓN

La Figura 1 muestra todos los servicios de AWS utilizados en Backend, Frontend y Data Science. Los elementos en rojo y trazo punteado corresponden a lo que sigue pendiente de desplegar o configurar.

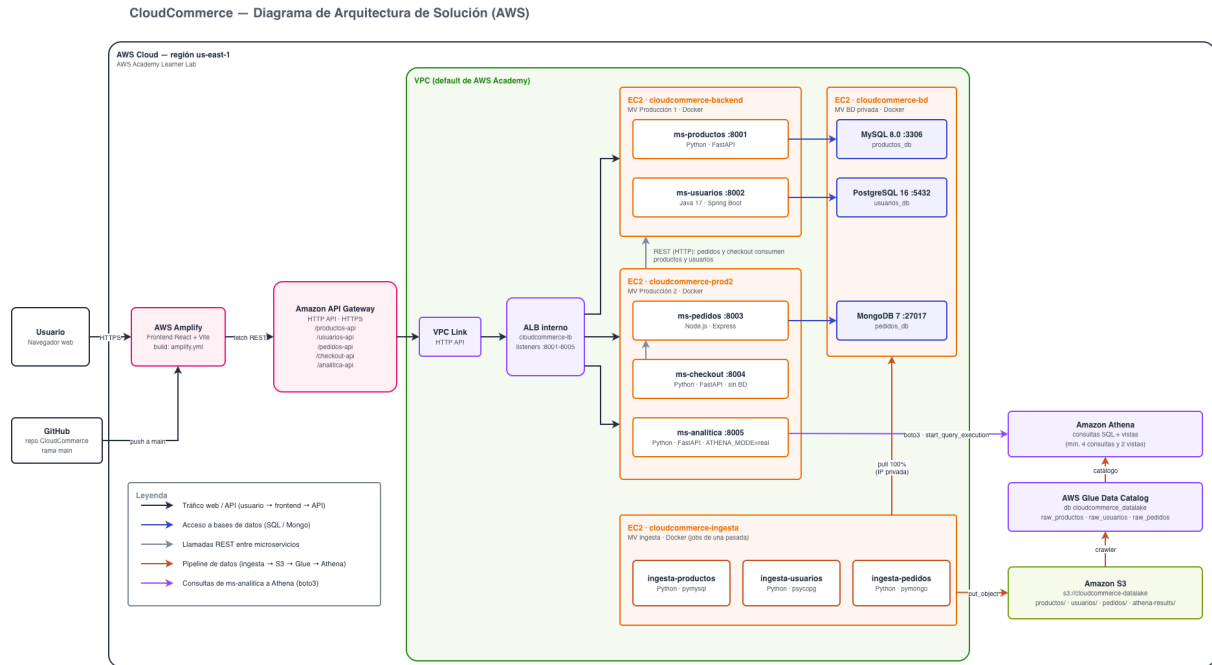


Figura 1. Diagrama de Arquitectura de Solución de CloudCommerce

2.1 Componentes y servicios AWS

Capa	Servicio / recurso	Función
Frontend	AWS Amplify	Hospeda la SPA React + Vite. Monorepo (appRoot: frontend), build definido en amplify.yml ; las URLs de la API se inyectan con variables VITE_*_API_URL .
Entrada	Amazon API Gateway (HTTP API)	Punto público HTTPS con CORS. Una ruta por microservicio: /productos-api , /usuarios-api , /pedidos-api , /checkout-api (y /analitica-api pendiente).
Red privada	VPC Link cloudcommerce-vpclink + ALB interno cloudcommerce-lb	API Gateway llega al balanceador por integración privada. El ALB no tiene IP pública y expone un listener por microservicio (8001–8004) con su target group y health check /health .
Cómputo	4 × Amazon EC2 (Ubuntu Server, Docker)	cloudcommerce-backend y cloudcommerce-prod2 (producción), cloudcommerce-bd (bases de datos) y cloudcommerce-ingesta (jobs de ingesta).

Datos transaccionales	MySQL 8, PostgreSQL 16, MongoDB 7 (contenedores en cloudcommerce-bd)	Una base por microservicio, en una MV privada sin IP pública.
Data lake	Amazon S3 cloudcommerce-datalake	Destino de la ingesta (prefijos productos/ , usuarios/ , pedidos/) y de los resultados de Athena.
Catálogo y consultas	AWS Glue Data Catalog + Amazon Athena	Base cloudcommerce_datalake con tablas, consultas SQL y vistas sobre los datos en S3.
Identidad	Rol IAM LabInstanceProfile	Credenciales de instancia para S3, Glue y Athena; no hay llaves en el código.

2.2 Recorrido de una petición

1. El usuario abre la aplicación servida por **Amplify**.
2. El navegador llama por HTTPS a **API Gateway** (por ejemplo, /pedidos-api/...).
3. La ruta usa una integración privada a través del **VPC Link** y llega al **ALB interno**.
4. El ALB reenvía según el puerto del listener al target group y a la EC2 correspondiente.
5. El microservicio consulta su base de datos por IP privada en cloudcommerce-bd.
6. ms-pedidos y ms-checkout consumen a ms-productos y ms-usuarios por HTTP para validar datos.

2.3 Decisiones de diseño y seguridad

- **Bases de datos aisladas:** viven en una tercera MV sin IP pública; sus security groups sólo admiten las IP privadas de las MV de producción y de ingesta.
- **Entrada única:** los puertos 8001–8004 de las MV de producción aceptan tráfico solo desde el security group del balanceador.
- **Ingesta desacoplada:** corre en su propia MV y lee las bases por IP privada, sin afectar la producción.
- **Datos sensibles:** la ingesta de usuarios no exporta password_hash.
- **Analítica desacoplada de AWS:** ms-analitica accede a Athena mediante un cliente abstraído con modo mock (por defecto) y real (ATHENA_MODE).

3. BACKEND: MICROSERVICIOS

El dominio es un e-commerce. Cada microservicio se ejecuta en un contenedor Docker, expone documentación Swagger/OpenAPI y responde /health para los health checks del balanceador.

"Código fuente de los 5 microservicios:
<https://github.com/DaniSandt1/CloudCommerce/tree/main/backend>"

3.1 Resumen

Microservicio	Tecnología	Base de datos	Puerto	Rol
ms-productos	Python 3.12 · FastAPI · SQLAlchemy	MySQL 8	8001	Catálogo de productos y categorías
ms-usuarios	Java 17 · Spring Boot 3.3	PostgreSQL 16	8002	Clientes y sus direcciones
ms-pedidos	Node.js 22 · Express 5 · Mongoose	MongoDB 7	8003	Órdenes de compra
ms-checkout	Python · FastAPI · httpx	Sin BD	8004	Orquesta productos + usuarios + pedidos
ms-analitica	Python · FastAPI · boto3	Athena (sin BD transaccional)	8005	Consultas analíticas sobre el data lake

Los tres servicios con base de datos se cargaron con 20 000 registros ficticios cada uno (Faker en productos, DataSeeder en usuarios y seedPedidos.js en pedidos). ms-pedidos consume a ms-productos y ms-usuarios, y ms-checkout consume a los otros tres, lo que cumple el requisito de comunicación entre microservicios.

3.2 ms-productos

Método	Ruta	Descripción
GET	/health	Estado y total de productos
GET / POST	/categorias	Lista y crea categorías
GET	/productos	Lista paginada (page, size) con filtros categoria_id y q (búsqueda por nombre)
GET	/productos/{id}	Detalle de un producto
POST	/productos	Crea un producto
PUT	/productos/{id}	Actualiza un producto
DELETE	/productos/{id}	Elimina un producto (204)

3.3 ms-usuarios

Método	Ruta	Descripción
GET	/health	Estado y total de usuarios
GET	/usuarios	Lista paginada con búsqueda q por nombre o email
GET / PUT / DELETE	/usuarios/{id}	Consulta, actualiza o elimina un usuario
POST	/usuarios	Crea un usuario (201). password_hash es solo de escritura: nunca se devuelve
GET / POST	/usuarios/{id}/direcciones	Lista y agrega direcciones del usuario

OpenAPI definition v0 OAS 3.0
/v3/api-docs

Servers

http://localhost:8002 - Generated server url

usuario-controller

GET /usuarios/{id}

PUT /usuarios/{id}

DELETE /usuarios/{id}

GET /usuarios

POST /usuarios

GET /usuarios/{id}/direcciones

POST /usuarios/{id}/direcciones

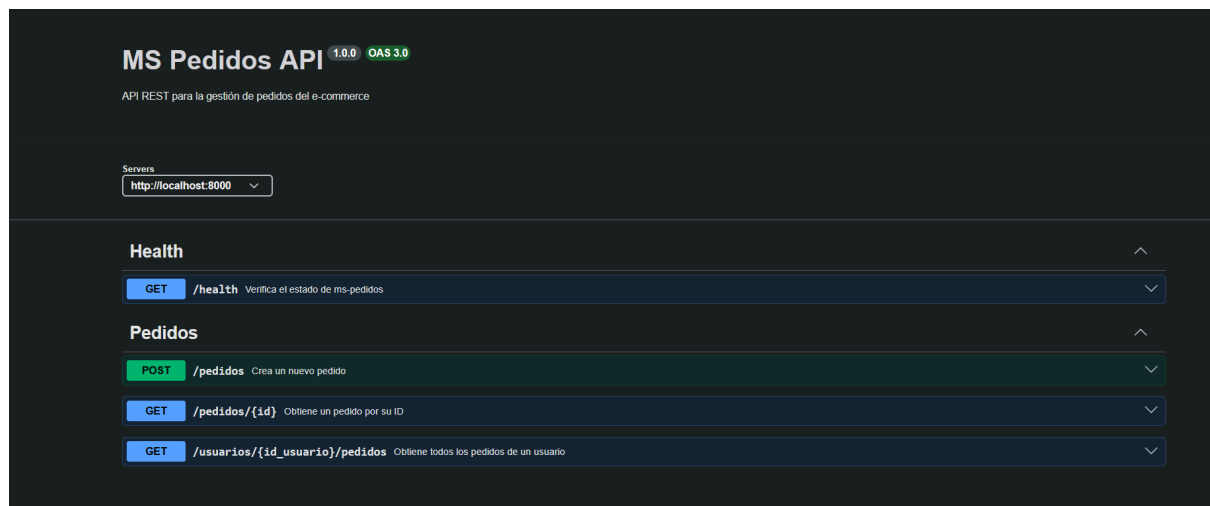
GET /health

GET /

3.4 ms-pedidos

Método	Ruta	Descripción
GET	/health	Estado del servicio
POST	/pedidos	Crea un pedido: valida que exista el usuario (ms-usuarios), toma nombre y precio de cada producto (ms-productos) y calcula el total. Estado inicial pendiente
GET	/pedidos/{id}	Detalle de un pedido por su ID de MongoDB
GET	/usuarios/{id_usuario}/pedidos	Pedidos de un usuario

El precio unitario queda congelado dentro de cada item, de modo que el historial no cambia si el producto se modifica luego. Estados posibles: pendiente, pagado, enviado y cancelado.



3.5 ms-checkout

Método	Ruta	Descripción
GET	/health	Estado del servicio (database: none)
POST	/checkout/resumen	Valida usuario, productos y stock y calcula el total, sin crear pedido
POST	/checkout/confirmar	Repite la validación y crea el pedido en ms-pedidos (201)
GET	/checkout/pedidos/{id_pedido}	Pedido enriquecido con datos del usuario y nombres de producto

ms-checkout ^{1.0.0} ^{OAS 3.1}

/openapi.json

Microservicio sin base de datos que orquesta ms-productos, ms-usuarios y ms-pedidos.

health		^
GET	/health Health	▼
checkout		^
POST	/checkout/resumen Valida y calcula el checkout sin crear un pedido	▼
POST	/checkout/confirmar Valida el checkout y crea el pedido mediante ms-pedidos	▼
GET	/checkout/pedidos/{id_pedido} Obtiene un pedido enriquecido con usuario y productos	▼
Schemas		^
CheckoutInput > Expand all object		
CheckoutItemInput > Expand all object		
CheckoutResumen > Expand all object		

3.6 ms-analitica

Método	Ruta	Descripción
GET	/health	Estado del servicio
GET	/analitica/ventas-por-categoria	Monto vendido por categoría de producto
GET	/analitica/top-clientes?limit=10	Clientes con mayor monto comprado (limit entre 1 y 100)
GET	/analitica/pedidos-por-rango?fecha_inicio=&fecha_fin=	Cantidad y monto de pedidos en un rango de fechas (formato YYYY-MM-DD , inclusivo)

CloudCommerce - ms-analitica ^{1.0.0} ^{OAS 3.1}

/openapi.json

Estado		^
GET	/health Health	▼
Analitica		^
GET	/analitica/ventas-por-categoria Ventas Por Categoria	▼
GET	/analitica/top-clientes Top Clientes	▼
GET	/analitica/pedidos-por-rango Pedidos Por Rango	▼
Schemas		^
HTTPValidationError > Expand all object		
PedidosPorRango > Expand all object		
TopCliente > Expand all object		

Index	categoria	total_vendido
0	Libros	19296367.95
1	Juguetes	19273937.38
2	Alimentos	19033472.58
3	Belleza	18853939.46
4	Deportes	18552237.94
5	Electrónica	18244989.19
6	Ropa	18234305.41
7	Hogar	18025029.21

3.7 Modelos de datos

Las Figuras 2 y 3 muestran el modelo entidad/relación de las bases SQL. La base de ms-pedidos es documental (MongoDB): los items van embebidos y las referencias a usuario y producto se guardan por ID.

Diagrama E/R — MySQL (productos_db · ms-productos)

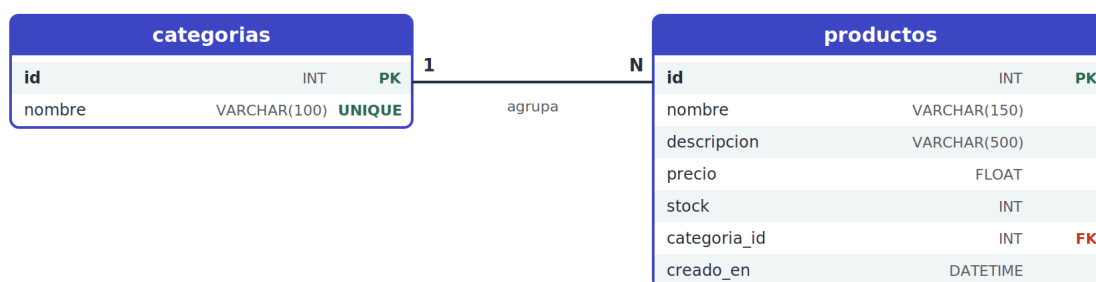


Figura 2. Diagrama E/R de MySQL (ms-productos).

Diagrama E/R — PostgreSQL (usuarios_db · ms-usuarios)

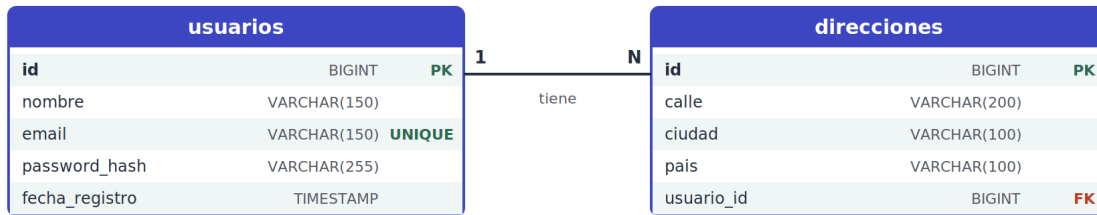


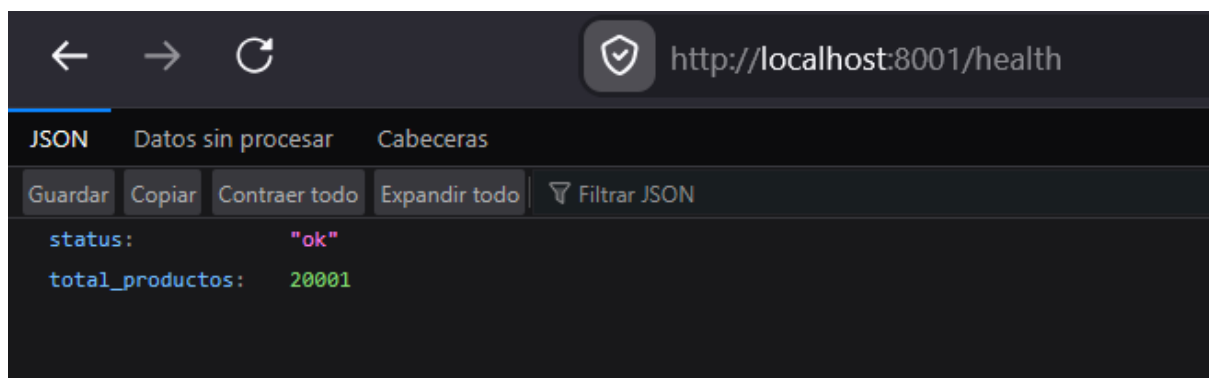
Figura 3. Diagrama E/R de PostgreSQL (ms-usuarios).

Estructura JSON de la colección ‘pedidos’ (MongoDB)

```
{
  "_id": "ObjectId",
  "id_usuario": 123,
  "fecha": "2026-09-10T10:00:00Z",
  "items": [
    { "id_producto": 45, "cantidad": 2, "precio_unitario": 19.90 }
  ],
  "total": 39.80,
  "estado": "pendiente"
}
```

3.8 Prueba y documentación de las API

- Pruebas unitarias en ms-checkout y ms-analitica (python -m unittest), sin depender de AWS ni de otros servicios.
- Swagger UI en cada microservicio y colecciones de Postman en ms-productos y ms-analitica.
- Carga de datos verificable por /health (total_productos, total_usuarios) y scripts/countPedidos.js.

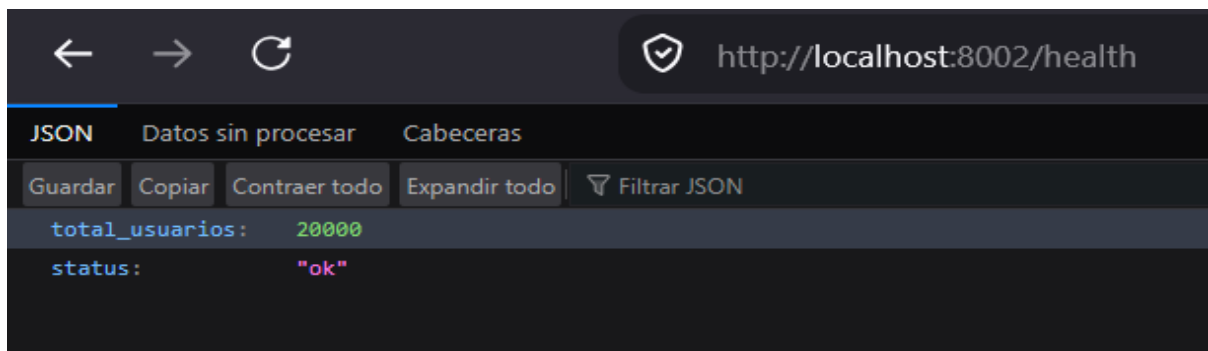


4. FRONTEND

"Código fuente del frontend:

<https://github.com/DaniSandt1/CloudCommerce/tree/main/frontend>"

La aplicación web usa React 18, Vite 5 y Tailwind CSS 3, con íconos de lucide-react, y está desplegada en AWS Amplify. Consume los cinco microservicios a través de API Gateway; las URLs base se configuran con variables de entorno (VITE_PRODUCTORS_API_URL, VITE_USUARIOS_API_URL, VITE_PEDIDOS_API_URL, VITE_CHECKOUT_API_URL y VITE_ANALITICA_API_URL)



```
Last login: Tue Sep 22 00:49:50 2026 from 45.236.45.48
ubuntu@ip-172-31-74-245:~$ cd ~/CloudCommerce/backend/docker-compose
ubuntu@ip-172-31-74-245:~/CloudCommerce/backend/docker-compose$ docker compose exec ms-pedidos node scripts/countPedidos.js
Cantidad de pedidos: 20000

Pedido más reciente:
{
  "id": "6aadd2c3649326e2cc0bb9df",
  "id_usuario": 16403,
  "fecha": "2026-09-19T00:09:39.222Z",
  "items": [
    {
      "id_producto": 14567,
      "cantidad": 4,
      "precio_unitario": 1507.27
    },
    {
      "id_producto": 834,
      "cantidad": 2,
      "precio_unitario": 207.1
    }
  ],
  "total": 6443.28,
  "estado": "pagado",
  "v": 0
}
ubuntu@ip-172-31-74-245:~/CloudCommerce/backend/docker-compose$
```

i-02e179efe1b736c20 (cloudcommerce-prod2)

Pestaña	Funcionalidad	Endpoints que invoca
Productos	Catálogo paginado con búsqueda por nombre, filtro por categoría, detalle en modal y alta de producto	GET /productos, GET /categorias, POST /productos
Usuarios	Listado paginado con búsqueda por nombre/email y alta de usuario	GET /usuarios, POST /usuarios
Pedidos	Búsqueda de pedidos por ID de usuario, detalle y flujo “Nuevo pedido”: primero resumen (precios y stock) y luego confirmación	GET /usuarios/{id}/pedidos, GET /pedidos/{id}, POST /checkout/resumen, POST /checkout/confirmar
Analítica	Ventas por categoría, top de clientes (5, 10, 20 o 50) y pedidos por rango de fechas	GET /analitica/*

Despliegue en Amplify

El repositorio de GitHub está conectado a Amplify como monorepo (appRoot: frontend). Cada push a main dispara el build (npm ci y npm run build) y publica el contenido de dist/.

The screenshot shows the 'Usuarios' page of the CloudCommerce application. The page features a grid of 18 user cards, each containing a circular profile picture, the user's name, email address, and a unique ID. The navigation bar at the top includes links for 'Productos', 'Usuarios' (which is currently selected), 'Pedidos', and 'Analítica'. A 'Nuevo usuario' button is located in the top right corner. The page footer indicates that there are 20,000 users in total and that this is page 1 of 1,000.

Initials	Name	Email	ID
SA	Sta. Inés Villagómez Ayala	teresa.carrera@hotmail.com.19999	ID: #20000
AO	Alfredo Godínez Ozuna	octavio.aguilar@yahoo.com.19998	ID: #19999
RA	Rosalía Juárez Atencio	marisol.montanez@gmail.com.19997	ID: #19998
DT	Diego Gallardo Terrazas	raul.cintron@hotmail.com.19996	ID: #19997
MT	Manuel Prieto Tirado	horacio.osorio@yahoo.com.19995	ID: #19996
RS	Rafael Salgado Sepúlveda	luz.griego@yahoo.com.19994	ID: #19995
BC	Beatriz Villanueva Castañeda	beatriz.trevino@hotmail.com.19993	ID: #19994
MA	Martín Barreto Adorno	jose.luis.giron@yahoo.com.19992	ID: #19993
SS	Sara Esparza Serna	pablo.armas@yahoo.com.19991	ID: #19992
JA	Julia Tello Aparicio	homero.arriaga@hotmail.com.19990	ID: #19991
BO	Barbara Cintrón Ocampo	mariano.marroquin@yahoo.com.19989	ID: #19990
CG	Claudia Márquez Gallardo	gabriel.peres@gmail.com.19988	ID: #19989
AD	Alejandra Murillo Dueñas	victor.solis@yahoo.com.19987	ID: #19988
DO	Diego Bustos Ornelas	antonio.navarro@gmail.com.19986	ID: #19987
MU	Mayte Orosco Ulibarri	elvira.banuelos@hotmail.com.19985	ID: #19986
CA	Carlota Narváez Álvarez	manuela.medina@yahoo.com.19984	ID: #19985
SV	Sr. Catalina Ortiz Vanegas	catalina.zamudio@gmail.com.19983	ID: #19984
GH	Gilberto Villalpando Holguín	emilia.narvaez@gmail.com.19982	ID: #19983

CloudCommerce

Productos

Usuarios

Pedidos

Analítica

1

×

Q

1 pedido del usuario #1

Nuevo pedido

Pedido #cc0b8410

13/9/2026 - 2 productos

Pagado

S/ 6843.11

CloudCommerce

Productos

Usuarios

Pedidos

Analítica

Ventas por categoría

Libros	S/ 19296367.95
Juguetes	S/ 19273937.38
Alimentos	S/ 19033472.58
Belleza	S/ 18853939.46
Deportes	S/ 18552237.94
Electrónica	S/ 18244989.19
Ropa	S/ 18234305.41
Hogar	S/ 18025029.21

Top clientes

Top 10

#1	Eduardo Ochoa Limón	mariaeugenia.villalpando@yahoo.com.13425	S/ 75732.04
#2	Cristobal Orozco Saldaña	cesar.nino@hotmail.com.2190	S/ 69757.46
#3	Gabriel Lucio Cotto	micaela.giron@yahoo.com.6364	S/ 63380.34
#4	...	...	...

Buscar

[Alt+S]

Estados Unidos (Norte de Virginia)

ID de cuenta: 1445-4872-4267
voclabs/user5310218=daniel.sandoval@utec.edu

Todas las aplicaciones > CloudCommerce: Información general

Proporcionar comentarios Soporte Documentos

CloudCommerce

ID de la aplicación: dh78atozp2fhn

Administrar entornos aislados Visitar URL implementada

Pasar a producción

0 de 3 pasos completados

1 Agregar un dominio personalizado

Utilice su propio dominio personalizado con HTTPS gratuito para proporcionar una URL segura y accesible para su aplicación.

Agregar dominio personalizado

2 Activar protecciones de firewall

AWS Web Application Firewall (WAF) ofrece restricciones de tráfico web para Amplify Hosting.

Activar firewall

3 Conectar nuevas ramificaciones

Conectar otra ramificación de su repositorio de Git para configurar varios entornos.

Conectar una nueva ramificación

Ramificaciones 1

Buscar...

+ Agregar ramificación

main

Implementación realizada

★ Ramificación de producción

Dominio

https://main.dh78atozp2fhn.amplifyapp.com

Última implementación

Hace 22 horas

Última confirmación

Ajustar ingesta de pedidos a J... / CloudCommerce:main

Todas las aplicaciones > CloudCommerce > Alojamiento: Variables de entorno

Proporcionar comentarios Soporte Documentos

CloudCommerce

Información general

Hospedaje

Control de acceso

Crear notificaciones

Configuración de compilación

Dominios personalizados

Caché y encabezados personalizados

Variables de entorno

Firewall

Vistas previas

Rescrituras y redireccionamientos

Secretos

Supervisión

Configuración de la aplicación

Configuración general

Variables de entorno

Administrar las variables

Las variables de entorno son pares de claves o valores que contienen cualquier valor constante que la aplicación necesite en el momento de la compilación, por ejemplo, los detalles de la conexión a la base de datos o las claves de la API de terceros.

Más información

Ramificación :	Variable :	Valor :
Todas las ramificaciones	AMPLIFY_DIFF_DEPLOY	false
Todas las ramificaciones	AMPLIFY_MONOREPO_APP_ROOT	frontend
Todas las ramificaciones	VITE_ANALITICA_API_URL	https://lb1rvjwotg.execute-api.us-east-1.amazonaws.com/analitica-api
Todas las ramificaciones	VITE_CHECKOUT_API_URL	https://lb1rvjwotg.execute-api.us-east-1.amazonaws.com/checkout-api
Todas las ramificaciones	VITE_PEDIDOS_API_URL	https://lb1rvjwotg.execute-api.us-east-1.amazonaws.com/pedidos-api
Todas las ramificaciones	VITE_PRODUCTOS_API_URL	https://lb1rvjwotg.execute-api.us-east-1.amazonaws.com/productos-api
Todas las ramificaciones	VITE_USUARIOS_API_URL	https://lb1rvjwotg.execute-api.us-east-1.amazonaws.com/usuarios-api

Rows per page 15 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

5. DATA SCIENCE

"Código fuente de la ingesta y el catálogo:

<https://github.com/DaniSandt1/CloudCommerce/tree/main/data-science>"

5.1 Ingesta hacia S3

Tres contenedores Docker en Python corren en la MV cloudcommerce-ingesta, separada de producción. Cada uno hace un pull del 100 % de su fuente y sube un archivo con marca de tiempo al bucket cloudcommerce-datalake. Son jobs de una sola pasada (docker compose run --rm <servicio>) y usan el rol de la instancia para acceder a S3.

Contenedor	Fuente	Formato	Destino en S3
ingesta-productos	MySQL (productos + categorías)	CSV	productos/productos_<timestamp>.csv
ingesta-usuarios	PostgreSQL (usuarios + direcciones)	CSV, una fila por dirección; password_hash	usuarios/usuarios_<timestamp>.csv
ingesta-pedidos	MongoDB (pedidos)	JSON Lines, un pedido por línea con items embebidos	pedidos/pedidos_<timestamp>.jsonl

cloudcommerce-datalake Información

Objetos (4)

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [Inventario de Amazon S3](#) para obtener una lista de todos los objetos de su bucket. Para que otras personas obtengan acceso a sus objetos, tendrá que concederles permisos de forma explícita. [Más información](#)

Buscar objetos por prefijo

Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
athena-results/	Carpeta	-	-	-
pedidos/	Carpeta	-	-	-
productos/	Carpeta	-	-	-
usuarios/	Carpeta	-	-	-

5.2 Catálogo de datos (AWS Glue)

Se creó la base cloudcommerce_datalake con una tabla por cada carpeta cargada a S3. En pedidos, el campo items se modela como array<struct<id_producto:bigint,cantidad:int,precio_unitario:double>> para poder expandirlo con UNNEST en Athena.

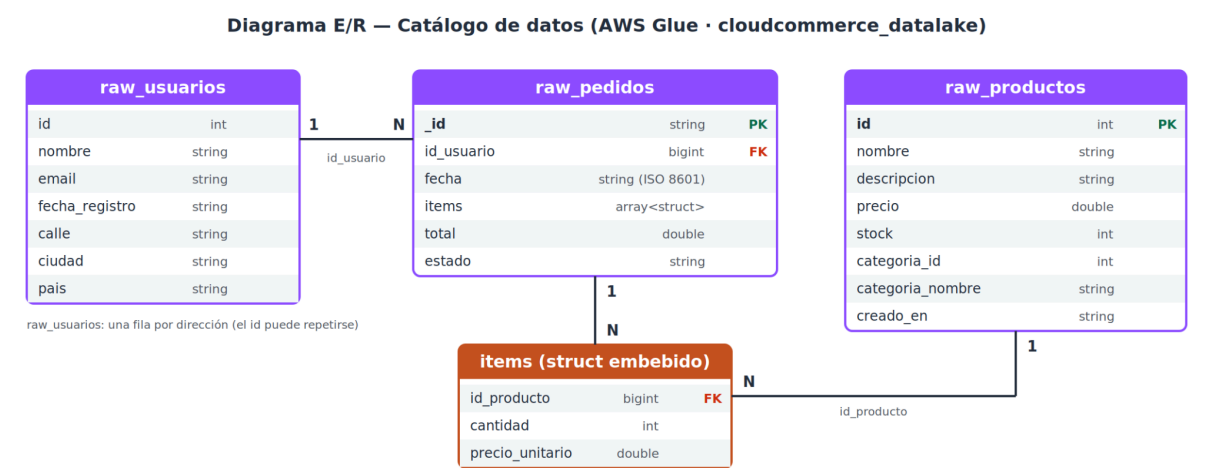


Figura 4. Diagrama E/R del catálogo de datos en AWS Glue.

Database properties

Name	Description	Location	Created on (UTC)
cloudcommerce_datalake	-	-	September 20, 2026 at 23:23:01

Tables (5)

Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
pedidos	cloudcommerce_datalake	s3://cloudcommerce-datalak	JSON	-	Table data	View data quality	View statistics
productos	cloudcommerce_datalak	s3://cloudcommerce-datalak	CSV	-	Table data	View data quality	View statistics
usuarios	cloudcommerce_datalake	s3://cloudcommerce-datalak	CSV	-	Table data	View data quality	View statistics
v_pedidos_actuales	cloudcommerce_datalake	-	-	-	-	View data quality	View statistics
v_productos_actuales	cloudcommerce_datalake	-	-	-	-	View data quality	View statistics

pedidos

Table details

Name: pedidos

Database: cloudcommerce_datalake

Description:

Last updated: September 20, 2026 at 23:23:14

Advanced properties

Schema (7)

#	Column name	Data type
1	_id	string
2	id_usuario	int
3	fecha	string
4	items	array
5	total	double
6	estado	string
7	...	int

productos

Table details

Name: productos

Database: cloudcommerce_datalake

Description:

Last updated: September 20, 2026 at 23:23:13

Advanced properties

Schema (8)

#	Column name	Data type
1	_id	bigint
2	nombre	string
3	descripcion	string
4	precio	double
5	stock	int
6	categoria_id	bigint
7	categoria_nombre	string
8	creado_en	string

usuarios

Table details

Name: usuarios

Database: cloudcommerce_datalake

Description:

Last updated: September 20, 2026 at 23:23:14

Advanced properties

Schema (7)

#	Column name	Data type
1	_id	bigint
2	nombre	string
3	email	string
4	fecha_registro	string
5	calle	string
6	ciudad	string
7	pais	string

5.3 Catálogo de datos (AWS Glue)

Como cada corrida de ingesta agrega un archivo nuevo, las tablas acumulan varias versiones del mismo dato. Por eso las dos vistas seleccionan sólo el último snapshot (MAX("\$path")) y las consultas parten de ellas. El CSV de usuarios trae una fila por dirección, así que se deduplica por id antes de unir.

Vista 1 — `v\_pedidos\_actuales`

```
CREATE OR REPLACE VIEW v_pedidos_actuales AS
SELECT _id, id_usuario, fecha, items, total, estado
FROM pedidos
WHERE "$path" = (SELECT MAX("$path") FROM pedidos);
```

Vista 2 — `v\_productos\_actuales`

```
CREATE OR REPLACE VIEW v_productos_actuales AS
SELECT id, nombre, precio, stock, categoria_id, categoria_nombre
FROM productos
WHERE "$path" = (SELECT MAX("$path") FROM productos);
```

The screenshot displays the AWS Glue console interface. On the left sidebar, under 'Datos', the 'Origen de datos' is set to 'AwsDataCatalog', 'Catálogo' is 'Ningún elemento', and 'Base de datos' is 'cloudcommerce_datalake'. The 'Tablas y vistas' section shows a list of tables ('pedidos', 'productos', 'usuarios') and views ('v_pedidos_actuales', 'v_productos_actuales'). The main panel shows a SQL query editor with four tabs: 'Consulta 1', 'Consulta 2', 'Consulta 3', and 'Consulta 4'. The active tab 'Consulta 1' contains the SQL code for creating the two views. Below the editor, there are buttons for 'Ejecutar', 'Explicar', 'Cancelar', 'Borrar', and 'Crear'. The 'Resultados de la consulta' tab is active, showing a green status bar with a checkmark and the text 'Completado'. Below this, a message states 'La consulta se ha realizado correctamente.'

Consulta 1 — Ventas por categoría (pedidos + productos)

```
SELECT p.categoria_nombre AS categoria,
       SUM(CAST(i.cantidad AS BIGINT)) AS unidades,
       ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS
total_vendido
FROM v_pedidos_actuales o
CROSS JOIN UNNEST(o.items) AS t(i)
JOIN v_productos_actuales p
  ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)
GROUP BY p.categoria_nombre
ORDER BY total_vendido DESC;
```

```
1  -- CONSULTA 1: ventas por categoría
2  -- Una pedidos (items desanidados) con productos.
3
4
5  SELECT p.categoria_nombre AS categoria,
6         SUM(CAST(i.cantidad AS BIGINT)) AS unidades,
7         ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS total_vendido
8  FROM v_pedidos_actuales o
9  CROSS JOIN UNNEST(o.items) AS t(i)
10 JOIN v_productos_actuales p
11    ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)
12 GROUP BY p.categoria_nombre
13 ORDER BY total_vendido DESC;
```

SQL Ln 13, Col 29

Ejecutar de nuevo

Explicar i²

Cancelar

Borrar

Crear

Resultados de la consulta

Estado de la consulta

Completado

Resultados (8)

Q Filas de búsqueda

#	▼	categoria	▼	unidades	▼	total_vendido
1		Libros		19023		1.929636795E7
2		Juguetes		19202		1.927393738E7
3		Alimentos		19208		1.903347258E7
4		Belleza		18994		1.885393946E7
5		Deportes		18656		1.855223794E7
6		Electrónica		18678		1.824498919E7
7		Ropa		18213		1.823430541E7
8		Hogar		18116		1.802502921E7

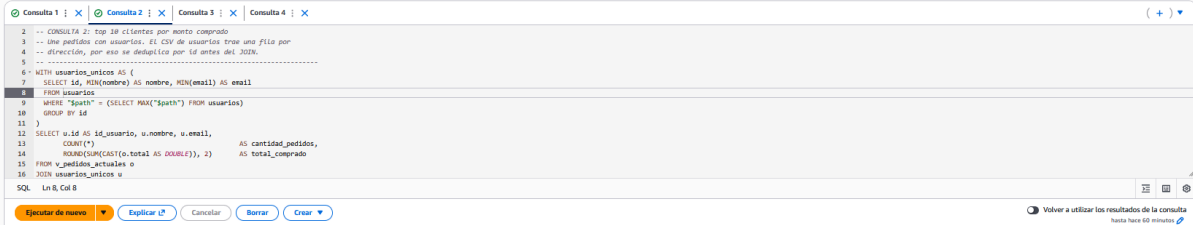
Consulta 2 — Top 10 clientes por monto (pedidos + usuarios)

```
WITH usuarios_unicos AS (
  SELECT id, MIN(nombre) AS nombre, MIN(email) AS email
  FROM usuarios
  WHERE "$path" = (SELECT MAX("$path") FROM usuarios)
  GROUP BY id
)
```

```

SELECT u.id AS id_usuario, u.nombre, u.email,
       COUNT(*) AS cantidad_pedidos,
       ROUND(SUM(CAST(o.total AS DOUBLE)), 2) AS total_comprado
FROM v_pedidos_actuales o
JOIN usuarios_unicos u
     ON CAST(u.id AS BIGINT) = CAST(o.id_usuario AS BIGINT)
GROUP BY u.id, u.nombre, u.email
ORDER BY total_comprado DESC
LIMIT 10;

```



The screenshot shows a SQL query editor with the following query:

```

2 -- CONSULTA 2: top 10 clientes por monto comprado
3 -- Una pedidos con usuarios. El CSV de usuarios trae una fila por
4 -- dirección, por eso se incluye por id antes del JOIN.
5 --
6 WITH usuarios_unicos AS (
7     SELECT id, MIN(nombre) AS nombre, MIN(email) AS email
8     FROM usuarios
9     WHERE "path" = (SELECT MAX("path") FROM usuarios)
10    GROUP BY id
11 )
12 SELECT u.id AS id_usuario, u.nombre, u.email,
13        COUNT(*) AS cantidad_pedidos,
14        ROUND(SUM(CAST(o.total AS DOUBLE)), 2) AS total_comprado
15 FROM v_pedidos_actuales o
16 JOIN usuarios_unicos u
17 SQL Ln 8, Col 8

```

The results section shows the following table:

#	id_usuario	nombre	email	cantidad_pedidos	total_comprado
1	13426	Eduardo Ochoa Lindón	mariaugenia.villalpando@yahoo.com.13425	5	75732.04
2	2191	Cristóbal Orozco Saldaña	cesar.nino@hotmail.com.2190	4	69757.46
3	6365	Gabriel Lucio Cotto	micaela.giron@yahoo.com.6364	7	63380.34
4	17305	Lorena Oquendo Romero	pablo.barriga@yahoo.com.17304	4	63084.07
5	17461	Sra. Fernando Valle Robles	josequin.sain@gmail.com.17460	5	60280.77
6	10092	Sra. Eva Armenta Arevalo	soledad.carbajal@gmail.com.10091	6	59403.1
7	3513	Sra. Carla Villarreal Cardona	gilberto.murillo@gmail.com.3512	4	59142.38
8	19208	Natalia Nazario Niño	maricristina.gomez@yahoo.com.19207	4	59107.09
9	799	Agustín Sedillo Agosto	javier.lopez@yahoo.com.798	4	58557.96
10	16830	Marisol Rangel Sierra	francisco.gallardo@yahoo.com.16829	3	57849.29

Consulta 3 — Top 10 productos más vendidos (pedidos + productos)

```

SELECT p.id AS id_producto, p.nombre, p.categoria_nombre AS categoria,
       SUM(CAST(i.cantidad AS BIGINT)) AS unidades_vendidas,
       ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)),
2) AS ingresos
FROM v_pedidos_actuales o
CROSS JOIN UNNEST(o.items) AS t(i)
JOIN v_productos_actuales p
     ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)
GROUP BY p.id, p.nombre, p.categoria_nombre
ORDER BY unidades_vendidas DESC

```

LIMIT 10;

1

--

2

-- CONSULTA 3: top 10 productos más vendidos

3

-- Une pedidos (items) con productos.

4

--

5

SELECT p.id AS id_producto, p.nombre, p.categoria_nombre AS categoria,

6

SUM(CAST(i.cantidad AS DOUBLE)) AS unidades_vendidas,

7

ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS Ingresos

8

FROM v_pedidos_actuales o

9

CROSS JOIN UNNEST(o.items) AS t(i)

10

JOIN v_productos_actuales p

11

ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)

12

GROUP BY p.id, p.nombre, p.categoria_nombre

13

ORDER BY unidades_vendidas DESC

14

LIMIT 10;

15

SQLLn 15, Col 1

Ejecutar de nuevo

Explicar [?]

Cancelar

Borrar

Crear ^v

Volver a utilizar los resu

tes

Resultados de la consulta

Estado de la consulta

Completado

Tiempo en cola: 63 ms

Tiempo de ejecución: 1.943 seg

Datos anali

Resultados (10)

Copiar

Descargar resultados

Q: Filas de búsqueda

#	id_producto	nombre	categoria	unidades_vendidas	Ingresos
1	9101	Customizable multimedia utilization	Juguetes	39	59087.34
2	13421	Fully-configurable didactic encryption	Electrónica	37	196.1
3	11559	Front-line web-enabled help-desk	Libros	33	23929.62
4	2511	Visionary needs-based toolset	Hogar	32	54912.64
5	9787	Centralized well-modulated task-force	Libros	32	45244.16
6	15749	Realigned directional infrastructure	Electrónica	32	31878.4
7	9934	Up-sized analyzing encoding	Libros	31	11875.79
8	17855	Profound global encoding	Hogar	31	60293.76
9	10595	Pre-emptive homogeneous core	Libros	31	14609.68
10	6335	Stand-alone maximized info-mediaries	Deportes	31	23997.72

Consulta 4 — Ventas por país y categoría (pedidos + productos + usuarios)

WITH usuarios_unicos AS (

SELECT id, MIN(pais) AS pais

FROM usuarios

WHERE "\$path" = (SELECT MAX("\$path") FROM usuarios)

GROUP BY id

)

SELECT u.pais, p.categoria_nombre AS categoria,

COUNT(DISTINCT o._id) AS pedidos,

ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS monto

FROM v_pedidos_actuales o

CROSS JOIN UNNEST(o.items) AS t(i)

JOIN v_productos_actuales p

ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)

JOIN usuarios_unicos u

ON CAST(u.id AS BIGINT) = CAST(o.id_usuario AS BIGINT)

GROUP BY u.pais, p.categoria_nombre

ORDER BY monto DESC

LIMIT 20;

```

2 -- CONSULTA 4: ventas por país del cliente y categoría
3 -- Une Los tres fuentes: pedidos + productos + usuarios.
4 .....
5 - WITH usuarios_unicos AS (
6   SELECT id, MD5(país) AS país
7   FROM usuarios
8 )
9 WHERE "spath" = (SELECT MAX("spath") FROM usuarios)
10 GROUP BY id
11 )
12 SELECT u.país, p.categoría_nombre AS categoría,
13        COUNT(DISTINCT o_id) AS pedidos,
14        ROUND(SUM(CAST(o.cantidad AS DOUBLE)) * CAST(i.precio_unitario AS DOUBLE)), 2) AS monto
15 FROM v_pedidos_actuales o
16 CROSS JOIN UNNEST(o.items) AS t(i)

```

SQL Ln 8, Col 45

Ejecutar de nuevo Explicar Cancelar Borrar Crear

Volver a utilizar los

Resultados de la consulta Estado de la consulta

Completado Tiempo en cola: 107 ms Tiempo de ejecución: 1.816 seg Datos

Resultados (20)

Filas de búsqueda

#	país	categoría	pedidos	monto
1	Costa Rica	Juergues	36	134979.58
2	Costa Rica	Deportes	40	134316.62
3	Guam	Deportes	26	133665.44
4	Estados Unidos de América	Juergues	31	128592.55
5	Costa Rica	Alimentos	33	126547.75
6	Haiti	Alimentos	29	125881.42
7	Huelva	Alimentos	29	125655.38
8	Eslovaquia	Alimentos	32	124560.68
9	Noruega	Libros	38	123764.13
10	Francia	Libros	31	123262.77
11	Alicante	Juergues	33	12045.82

6. DESPLIEGUE EN AWS

El despliegue se realizó en AWS Academy Learner Lab siguiendo DEPLOYMENT.md. La arquitectura final separa producción, bases de datos e ingesta en cuatro instancias EC2 dentro de la misma VPC.

Instancia	Rol	Contenido	Acceso de red
cloudcommerce-backend	Producción 1	ms-productos :8001, ms-usuarios :8002	Puertos de aplicación solo desde el SG del balanceador
cloudcommerce-product2	Producción 2	ms-pedidos :8003, ms-checkout :8004 (y ms-analitica :8005 cuando se despliegue)	Puertos de aplicación solo desde el SG del balanceador
cloudcommerce-database	Bases de datos (privada)	MySQL :3306, PostgreSQL :5432, MongoDB :27017	Sin IP pública ni 0.0.0.0/0; solo IP privadas de producción e ingesta
cloudcommerce-ingesta	Data Science	ingesta-productos, ingesta-usuarios, ingesta-pedidos	Rol LabInstanceProfile (S3)

Pasos principales

- 1. Lanzar las instancias EC2 (Ubuntu Server, t3.medium en backend y BD), instalar Docker y clonar el repositorio.
- 2. Levantar las bases de datos en cloudcommerce-bd y los microservicios en las dos MV de producción con docker compose; cargar los datos ficticios.
- 3. Crear el ALB interno con un listener y target group por microservicio y verificar que estén *Healthy*.
- 4. Crear el VPC Link y las rutas de API Gateway con integración privada y CORS.
- 5. Publicar el frontend en Amplify con las URLs de API Gateway como variables de entorno.
- 6. Configurar la ingesta con las IP privadas de la MV de BD y ejecutar los tres jobs.

Instancias (4/9) Información

Última actualización Hace less than a minute

Conectar Estado de la instancia Acciones Lanzar instancias

Conjuntos de filtros guardados

Eligir conjunto de filtr...

Filtrar instancia por atributo o etiqueta (distingue mayúsculas y minúsculas)

	Nombre	ID de la instancia	Estado de la instancia	Tipo de instancia	Comprobación de salud	Zona de disponibilidad	DNS de IPv4 pública	Dirección IP pública	IP elástica	Direcciones IP privadas	Monitoreo	Nombre del grupo de usuarios
☐	MV Desarrollo	i-067603f0d0e0e0b0c	En ejecución	t3.micro	3/3 comprobador	us-east-1c	ec2-34-193-3-205.com...	34.193.3.205	34.193.3.205	-	disabled	launch-wizard
☒	cloudcommerce-bd	i-0098f30fc145a65bc	En ejecución	t3.medium	3/3 comprobador	us-east-1c	ec2-3-80-67-210.comp...	3.80.67.210	-	-	disabled	BD
☐	MV Pruebas	i-0bd5d9d43576d1cb1	En ejecución	t2.micro	2/2 comprobador	us-east-1b	ec2-5-87-0-111.comput...	5.87.0.111	-	-	disabled	crear-mv-prue
☒	cloudcommerce-backend	i-0142e0b53f7de2410	En ejecución	t3.medium	3/3 comprobador	us-east-1b	ec2-34-203-18-199.co...	34.203.18.199	34.203.18.199	-	disabled	launch-wizard
☐	MV Base de Datos	i-08f0fb89fad03ad35	En ejecución	t2.micro	2/2 comprobador	us-east-1b	ec2-54-85-143-5.comp...	54.85.143.5	-	-	disabled	crear-mv-bd-l
☐	MV-Prod 2	i-074db56d643ff4570	En ejecución	t3.micro	3/3 comprobador	us-east-1b	ec2-44-201-143-200.co...	44.201.143.200	-	-	disabled	G5-Prod
☐	MV Prod 1	i-051884c4399e329c7	En ejecución	t3.micro	3/3 comprobador	us-east-1a	ec2-100-60-80-251.co...	100.60.80.251	-	-	disabled	G5-Prod
☒	cloudcommerce-prod2	i-02e179efe1b736c20	En ejecución	t3.small	3/3 comprobador	us-east-1f	ec2-100-54-90-84.com...	100.54.90.84	-	-	disabled	launch-wizard
☒	cloudcommerce-ingesta	i-05655ec9519678a3f	En ejecución	t3.micro	3/3 comprobador	us-east-1c	ec2-3-93-31-85.comput...	3.93.31.85	-	-	disabled	launch-wizard

Grupos de destino (5/6) Info Novedades

Filtrar grupos de destino

	Nombre	ARN	Puerto	Protocolo	Tipo de destino	Balanceador de carga	ID de VPC
☐	TG-Prod80	arn:aws:elasticloadbalancin...	80	HTTP	Instancia	lb-prod	vpc-069c9f21ded4c1ebc
☒	tg-analitica	arn:aws:elasticloadbalancin...	8005	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-checkout	arn:aws:elasticloadbalancin...	8004	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-pedidos	arn:aws:elasticloadbalancin...	8003	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-productos	arn:aws:elasticloadbalancin...	8001	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-usuarios	arn:aws:elasticloadbalancin...	8002	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc

Rutas

Etapas: Implementación

Rutas para cloudcommerce-api

Crear

Buscar

▼ /analitica-api

▼ /(proxy+)

ANY

▼ /checkout-api

▼ /(proxy+)

ANY

▼ /pedidos-api

▼ /(proxy+)

ANY

▼ /productos-api

▼ /(proxy+)

ANY

▼ /usuarios-api

▼ /(proxy+)

ANY

Detalles de la ruta

Eliminar Editar

ANY /analitica-api/(proxy+) (ID: 3rsz72)

ARN

arn:aws:apigateway:us-east-1:apis/lb1vjwotg/routes/3rsz72

Autorización

Los autorizadores protegen la API frente a las solicitudes no autorizadas. Las rutas sin autorizaciones asociadas son abiertas.

No hay ningún autorizador asociado a esta ruta.

Asociar autorización

Integración

La integración es el recurso de backend al que llama esta ruta cuando recibe una solicitud.

hqlz0vd

Configurar

Integraciones

Etapas: - **Implementación**

Asociar integraciones a rutas

Administrar integraciones

Rutas para cloudcommerce-api

Q Buscar

▼ /analitica-api

▼ /(proxy+)

ANY VPC Load Balancer

▼ /checkout-api

▼ /(proxy+)

ANY VPC Load Balancer

▼ /pedidos-api

▼ /(proxy+)

ANY VPC Load Balancer

▼ /productos-api

▼ /(proxy+)

ANY VPC Load Balancer

▼ /usuarios-api

▼ /(proxy+)

ANY VPC Load Balancer

Detalles de integración para la ruta

Desconectar integración

Gestionar integración

ANY /analitica-api/(proxy+) (3rzsz72)

Agente de escucha del balanceador de carga

ANY HTTP:8005 - cloudcommerce-lb L*

ID de integración

hgz0vd

Descripción

-

Enlace de VPC

[fg5wb](#)

Tiempo de espera

El número de milisegundos que API Gateway debe esperar por una respuesta de la integración antes de que se agote el tiempo de espera.

30000

Solicitar asignación de parámetros

Todas las solicitudes - Modificación 1

Asignación de parámetros de respuesta

No configurado

- **Seguridad de la aplicación:** las API no tienen autenticación y ms-usuarios no expone un login ni aplica hash a la contraseña recibida. Se propone incorporar autenticación con tokens y hash de contraseñas.
- **Ciclo del pedido:** el stock se valida en el checkout pero no se descuenta, y el estado del pedido no cambia después de pendiente. Se propone agregar actualización de stock y transiciones de estado.
- **Data lake:** cada ingesta genera un snapshot completo en el mismo prefijo; las vistas de Athena seleccionan el último. Una mejora es particionar por fecha para reducir lo que Athena escanea.

- La arquitectura en microservicios permitió combinar tres lenguajes y tres motores de base de datos, cada uno adecuado a su dominio.
- Aislar las bases en una MV privada y exponer todo mediante API Gateway, VPC Link y un ALB interno reduce la superficie de ataque.
- El pipeline S3 + Glue + Athena separa el análisis de la carga transaccional y habilita consultas que unen las tres fuentes.
- Abstractar el cliente de Athena (modo mock y real) permitió desarrollar y probar ms-analítica sin depender de credenciales de AWS.